# Barometric Pipeline

## Introduction

## Setup

This vignette assumes you have already installed `waterlogged` from the develop branch:

```
install.packages("pak")
pak::pkg_install("dfo-pacific-science/waterlogged@develop")
```

```
library(waterlogged)
```

## Prepare Raw Data

In most cases, there will be data for multiple loggers or downloads for a single station. These need to be bound together to create a single data file for each station.

Also, data needs to be trimmed to the install and removal dates in case loggers were activated prior to install, or if they were removed before downloading the data (which is typically the case).

To do this, you will need 2 things:

1. Folder containing raw .csv files exported from HOBOware (*baro_folder*)
    - we recommend keeping csv files from different loggers separate in different folders, and organizing logger folders by year and processing-stage. ie. a file path to the raw water-level logger data would be: "data/waterlevel/2024/raw"
2. A .csv of metadata (*metadata_file*) containing:
    - *sn* serial number of logger
    - *site_station_code* A unique ID for the station where the logger was deployed
    - *timestamp_deployed* timestamp of logger deployment
    - *timestamp_removed* timestamp of logger removal
    - *status* status of logger (logging, lost, removed)

Define paths to the test data.

```
metadata_file  <- system.file("extdata/metadata/metadata.csv", package = "waterlogged")
baro_folder    <- system.file("extdata/barometric_pressure", package = "waterlogged")
log_root_folder <- tempdir()
```

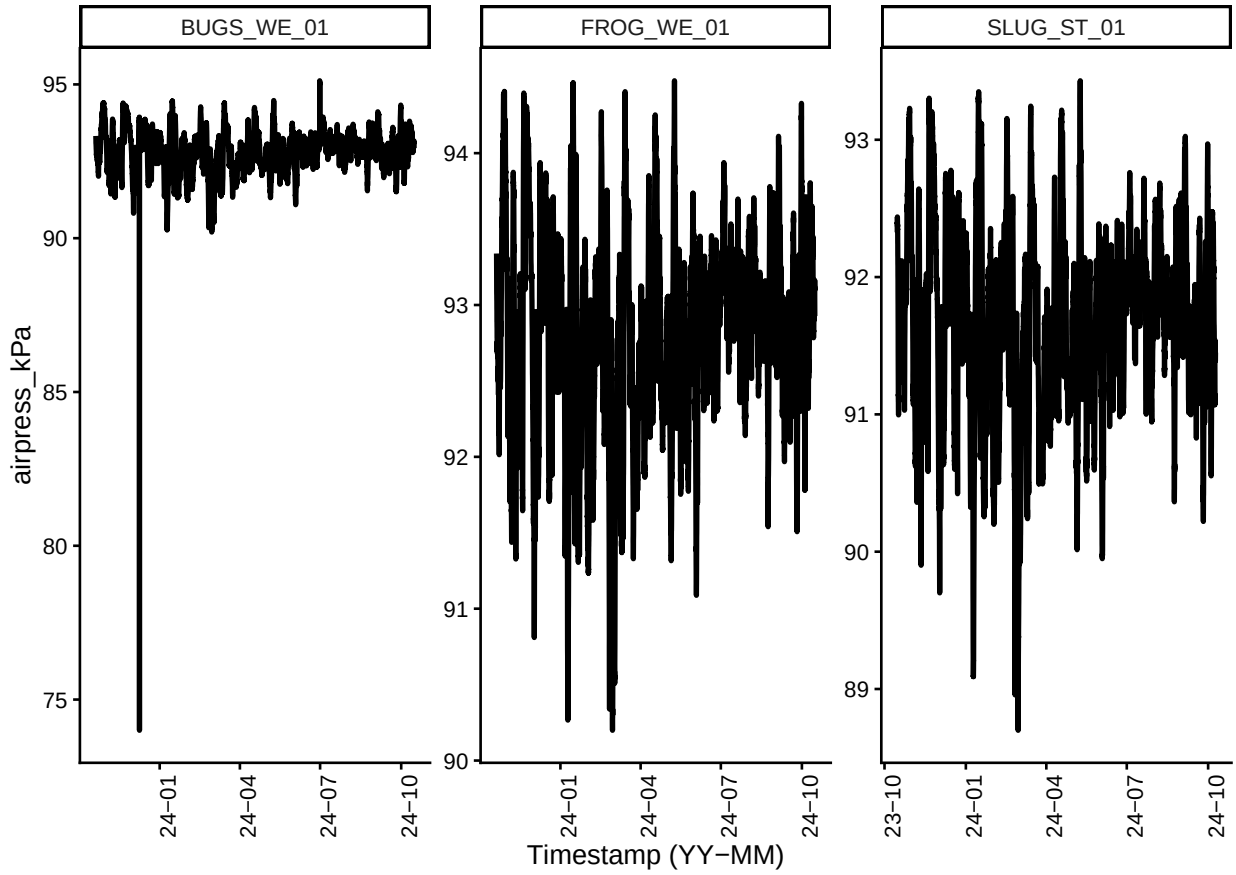
The function **bind_hobo_files** will load all .csv files in the *path_to_raw_folder*, match loggers with the *metadata* file, trim logger data by to only those timestamps within install/remove dates, and bind data from loggers by each station/location and year. The raw folder must ONLY contain raw logger CSV files of the same metric (ie. all logger files are from barometric pressure loggers).

To save bound files, specify the path to the desired output folder with the optional variable *path_to_output_folder*.

```
baro_bound <- bind_hobo_files(
  path_to_raw_folder = baro_folder,
  metadata_path      = metadata_file,
)

# to see the trimmed and re-bound baro dataset, save the first element of the list as a dataframe:
baro_data <- baro_bound[[1]]

# to see summary plot
baro_bound[[2]]
```




```
# summarize the trimmed and bound data
cat("Baro data:", nrow(baro_data), "rows,",
    length(unique(baro_data$site_station_code)), "stations\n")
#> Baro data: 25978 rows, 3 stations
```

Now we have individual files for barometric data at each station and year saved in their respective folders, and a trimmed dataset of all input baro files saved in the environment as **baro_data**.

## Automated Quality Control Protocols

Runs dataset through automated data outlier removal and flags, determined by user-modifiable thresholds. The default thresholds are as follows:

- temp_high_limit: Temperature readings above this value will be removed and data will be flagged as OUTLIER in flag_code column. Default is 50°C
- temp_low_limit: Temperature readings below this value will be removed and data will be flagged as OUTLIER in flag_code column. Default is -40°C.
- pressure_low_limit: Pressure readings below this value will be removed and flagged as OUTLIER in flag_code column. Default is 85 kPa.
- pressure_high_limit: Pressure readings above this value will be removed and flagged as OUTLIER in flag_code column. Default is 105 kPa.
- spike_threshold: The maximum allowable difference in pressure between consecutive readings. Labeled as SPIKE in data_flag column. Default is 1.5 kPa.
- flatline_n: The number of consecutive identical pressure readings that will result in a data_flag label of FLATLINE.

The optional input variable *log_root* is the path to the desired folder where a CSV file of all QC flags and adjustments are stored, by logger station.

```
baro_qc <- qc_barometric(
  input_data = baro_data,
  log_root = log_root_folder,
  select_station = unique(baro_data$site_station_code)[1]
)

cat("Baro QAQC complete:", nrow(baro_qc), "rows\n")
#> Baro QAQC complete: 8668 rows
cat("  Air Press. outliers:     ", sum(baro_qc$OUTLIER_AP, na.rm = TRUE), "\n")
#>   Air Press. outliers:      0
cat("  Air Temp. outliers:      ", sum(baro_qc$OUTLIER_AT, na.rm = TRUE), "\n")
#>   Air Temp. outliers:       4
cat("  Disturbances:            ", sum(baro_qc$FLAG_DISTURBANCE, na.rm = TRUE), "\n")
#>   Disturbances:             5
cat("  Flatlines:               ", sum(baro_qc$FLAG_FLATLINE,   na.rm = TRUE), "\n")
#>   Flatlines:                16

head(baro_qc, 3)
#> # A tibble: 3 x 19
#>   sn    timestamp           airpress_kPa airtemp_C file_name timezone data1_type
#>   <chr> <dttm>                     <dbl>     <dbl> <chr>     <chr>    <chr>
#> 1 2718~ 2023-10-20 13:00:00         93.3      21.4 raw_data~ GMT-07:~ Abs Pres
#> 2 2718~ 2023-10-20 14:00:00         93.3      21.1 raw_data~ GMT-07:~ Abs Pres
#> 3 2718~ 2023-10-20 15:00:00         93.2      16.0 raw_data~ GMT-07:~ Abs Pres
#> # i 12 more variables: data1_unit <chr>, data2_type <chr>, data2_unit <chr>,
#> #   site_station_code <chr>, logger_type <chr>, metric <chr>,
#> #   airpress_kPa_adj <dbl>, airtemp_C_adj <dbl>, OUTLIER_AP <lgl>,
#> #   FLAG_FLATLINE <lgl>, FLAG_DISTURBANCE <lgl>, OUTLIER_AT <lgl>
```

Note that data labeled as OUTLIER have been removed, while data labeled as FLAG is retained for user verification. The QC log produced will look like this:

```
system.file("extdata/qaqc_logs/BUGS_WE_01_BARO_qaqc_log.csv", package = "waterlogged")
#> [1] "C:/Users/mattj/AppData/Local/R/win-library/4.5/waterlogged/extdata/qaqc_logs/BUGS_WE_01_BARO_qa
```

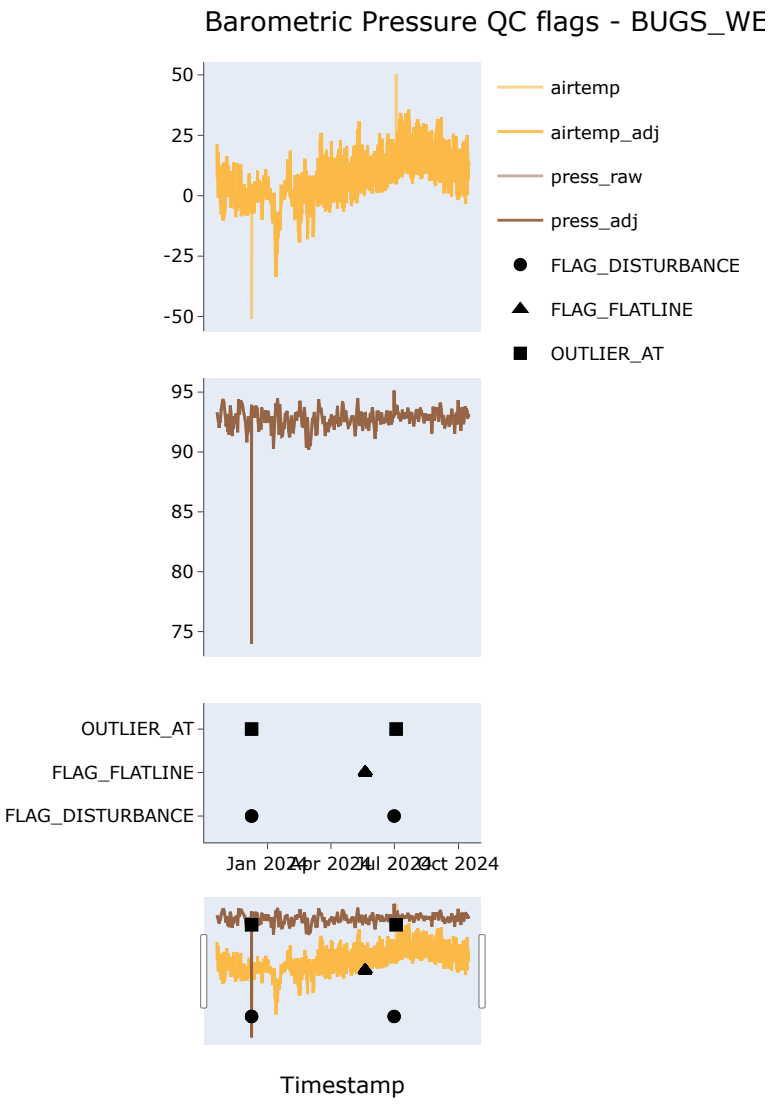
### Visual Review

Creates an interactive multi-panel plot for reviewing the data and either accepting or rejecting automated data flags.

How to read the plot:

- Light traces = raw values (original, unmodified)
- Dark traces = adjusted values (after auto QAQC)
- Black symbols = flagged data (ice, dry, error)
- Use the range slider at the bottom to zoom into specific periods
- Hover over points for exact values and timestamps

```
plot_qaqc_timeseries(
  baro_data      = baro_qc,
  select_station = unique(baro_data$site_station_code)[1]
)
```


Barometric Pressure QC flags - BUGS_WE
50
25
0
-25
-50
airtemp
airtemp_adj
press_raw
press_adj
FLAG_DISTURBANCE
FLAG_FLATLINE
OUTLIER_AT
95
90
85
80
75
OUTLIER_AT
FLAG_FLATLINE
FLAG_DISTURBANCE
Jan 2024
Apr 2024
Jul 2024
Oct 2024
Timestamp

## Accept or reject data flags

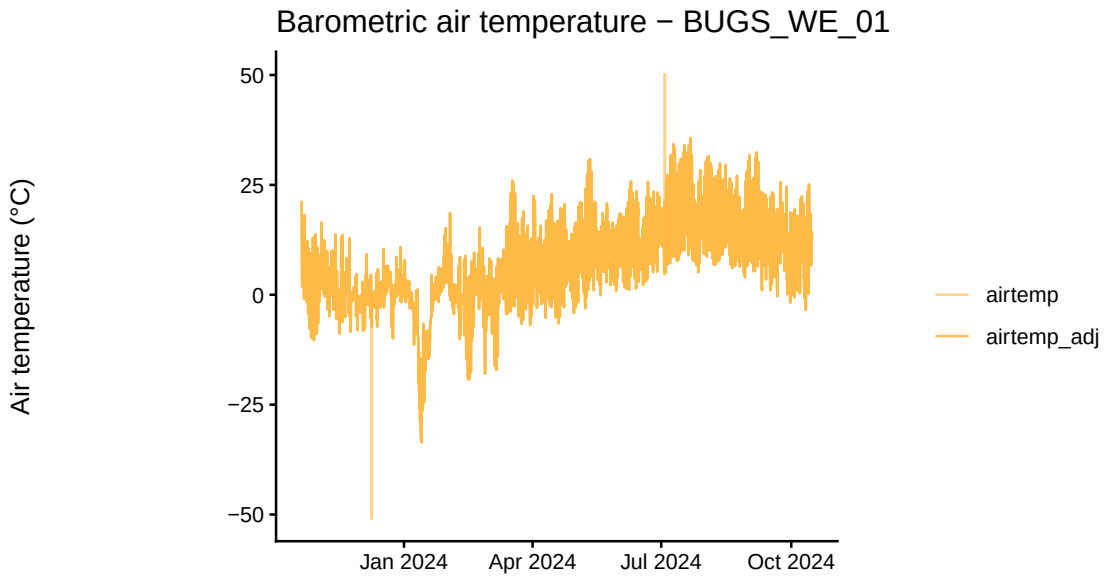
Inspect pressure data flagged as flatline in May by click and dragging over the triangle symbol (FLAG_FLATLINE) in the third plot. By hovering your mouse over the first triangle symbol you can extract the timestamp for when the flatline began: "2024-05-19 19:00:00". Likewise, find the timestamp where the flatline ended: "2024-05-20 10:00:00". Both temperature and air pressure have constant values for over 12 consecutive hours - this is likely a logging error. We can accept the flag and remove this data using the function *adjust_logger_NA*. Reason to adjust must be "disturbance" for barometric loggers. A manual note is required to note the reason for data removal, and is stored in the QC log file (within the designated log_root folder).

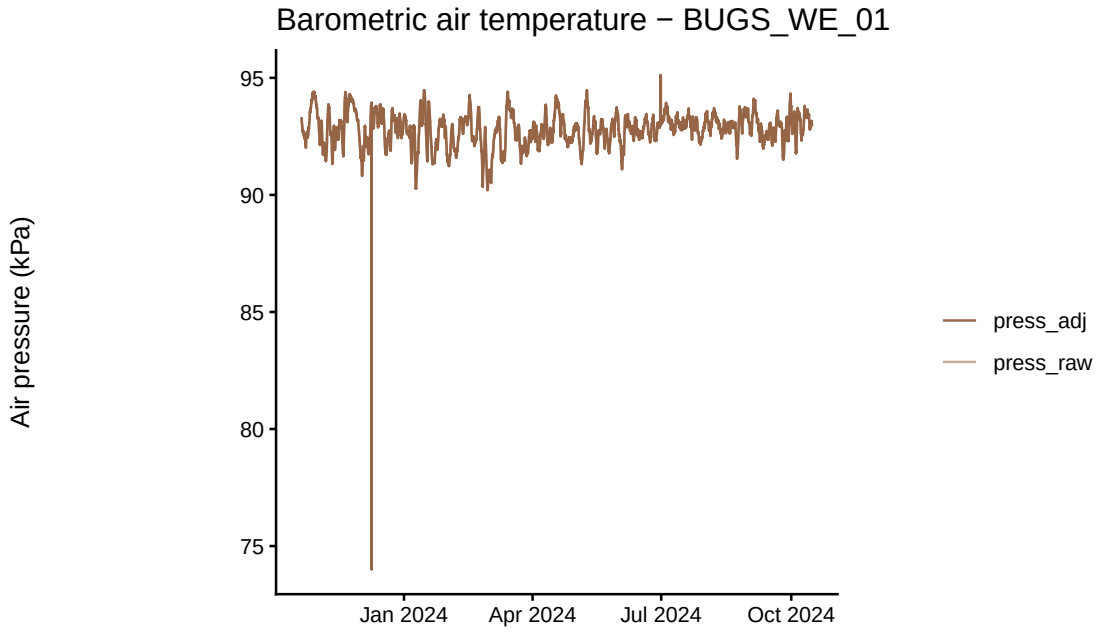
```
baro_qc <- adjust_logger_NA(
  input_data       = baro_qc,
  select_station   = unique(baro_data$site_station_code)[1],
  metric           = "barometric",
  reason_to_adjust = "disturbance",
  timestamp_start  = "2024-05-19 19:00:00",
  timestamp_end    = "2024-05-20 10:00:00",
  manual_note      = "Sensor error causing flatline",
  log_root = log_root_folder
)
```

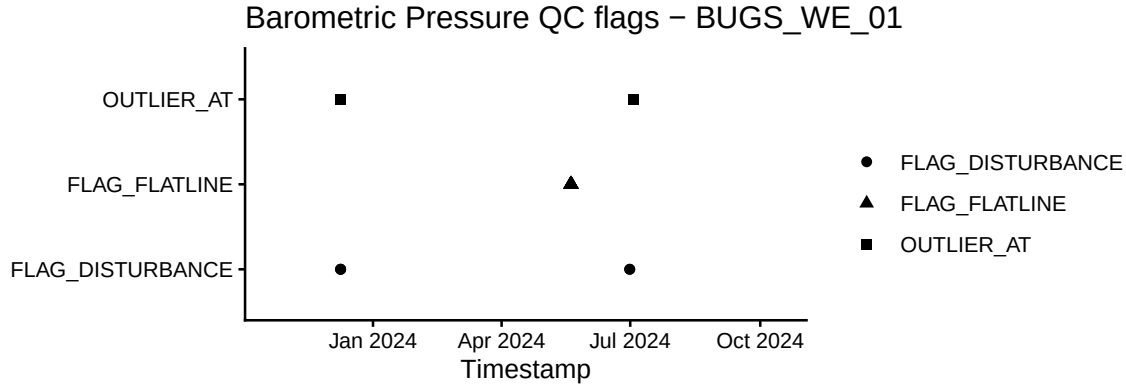
## Final Review

Re-plot the data after all corrections to verify the final result. The adjusted traces should look reasonable with no obvious artifacts. Compare raw (light) vs adjusted (dark) to see exactly what changed.

```
plot_qaqc_timeseries(
  baro_data      = baro_qc,
  select_station = unique(baro_data$site_station_code)[1],
  static=TRUE
)
```


Barometric air temperature − BUGS_WE_01
Air temperature (°C)
50
25
0
−25
−50
Jan 2024
Apr 2024
Jul 2024
Oct 2024
airtemp
airtemp_adj


Barometric air temperature − BUGS_WE_01
Air pressure (kPa)
95
90
85
80
75
Jan 2024
Apr 2024
Jul 2024
Oct 2024
press_adj
press_raw


Barometric Pressure QC flags − BUGS_WE_01
OUTLIER_AT
FLAG_FLATLINE
FLAG_DISTURBANCE
Jan 2024
Apr 2024
Jul 2024
Oct 2024
Timestamp
FLAG_DISTURBANCE
FLAG_FLATLINE
OUTLIER_AT

## Export Clean Data (v1.0)

Writes the final cleaned data to CSV files, one per station per year.

What gets exported:

- `site_station_code`, `timestamp` (identifiers)
- `airpress_kPa`, `airpress_kPa_adj` (raw and adjusted air pressure in kPa)
- `airtemp_C`, `airtemp_C_adj` (raw and adjusted air temperature)
- All `FLAG_*`, `OUTLIER_*` columns (the full audit trail)

```
cat("Final dataset:", nrow(baro_qc), "rows x", ncol(baro_qc), "columns\n\n")
#> Final dataset: 8668 rows x 19 columns

cat("Summary of raw AP (kPa):\n")
#> Summary of raw AP (kPa):
print(summary(baro_qc$airpress_kPa))
#>    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
#>   74.00   92.40   92.86   92.80   93.24   95.12

cat("Summary of adjusted AP (kPa):\n")
#> Summary of adjusted AP (kPa):
print(summary(baro_qc$airpress_kPa_adj))
#>    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
#>   74.00   92.40   92.86   92.80   93.24   95.12

cat("\nSummary of raw AT (degrees C):\n")
#>
#> Summary of raw AT (degrees C):
print(summary(baro_qc$airtemp_C))
#>    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
#> -51.000  -0.213   5.450   6.200  12.304  50.234

cat("\nSummary of adjusted AT (degrees C):\n")
#>
#> Summary of adjusted AT (degrees C):
print(summary(baro_qc$airtemp_C_adj))
#>    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's
#> -33.574  -0.213   5.450   6.201  12.304  35.649       4
```

```
write_clean_data(
  input_data            = baro_qc,
  path_to_output_folder = output,
  metric                = "barometric"
)
```